

SmartMeds: An Intelligent Voice-Driven IoT Medicine Dispenser with NLP-Based Intent Recognition for Enhanced Medication Adherence

Sumukha M L

Department of Computer Applications, PES University,
Bengaluru – 560085, Karnataka, India
PES1PG24CA184 | Guide: Mr. Santosh Katti, Assistant
Professor

Abstract

Medication non-adherence remains one of the most persistent and costly challenges in modern healthcare, contributing to preventable hospitalisations, disease progression, and increased mortality, especially among elderly and chronically ill populations. Traditional reminder-based approaches notify patients but cannot verify or enforce correct medicine intake. This paper presents SmartMeds, a cloud-integrated intelligent medicine dispenser that combines Natural Language Processing (NLP), Internet of Things (IoT), and mobile technologies to automate and validate the entire medication management workflow. The proposed system comprises a React Native (Expo) mobile application for both caregivers and patients, a Python Flask backend embedding a spaCy NLP engine for voice command parsing, a Firebase Realtime Database as the cloud coordination layer, and a Raspberry Pi 3 embedded controller that drives a 28BYJ-48 stepper motor for a 24-compartment rotating drum, supplemented by IR sensors, and a speaker as buzzer. Patients issue natural spoken commands; the system transcribes audio to text, extracts intent and medicine entities, validates the request against the stored prescription, and the Raspberry Pi autonomously rotates the drum to the correct slot while providing multimodal feedback via LCD display, visual indicators, and text-to-speech audio. A slot-based multi-medicine scheduling model allows each time slot to hold multiple medications, and an offline SQLite queue ensures Firebase write durability during connectivity loss. Eighteen structured test cases covering hardware actuation, cloud synchronisation, NLP parsing, tamper detection, and fault-handling all yielded passing results. Comparative analysis against five existing solution categories confirms that SmartMeds uniquely unifies voice interaction,

NLP intent recognition, automated physical dispensing, mobile caregiver supervision, tamper detection, and offline fault tolerance within a low-cost open-source architecture.

Keywords—medication adherence, IoT medicine dispenser, natural language processing, Raspberry Pi, voice-controlled healthcare, spaCy, Firebase, React Native, smart health systems

I. Introduction

Adherence to prescribed medication regimens is fundamental to the effectiveness of pharmacological therapy. The World Health Organization estimates that medication non-adherence accounts for approximately 50% of treatment failures in chronic disease management and is a contributing factor in nearly 125,000 preventable deaths annually in the United States alone [1]. Elderly patients, individuals managing multiple chronic conditions, and those with cognitive or motor impairments are disproportionately affected, as the complexity of polypharmacy schedules and the physical difficulty of distinguishing similarly packaged pills compound the risk of missed or incorrect doses.

Current technological interventions span a spectrum from simple mobile reminders to commercially automated dispensers. Mobile reminder applications alert patients at scheduled times but rely entirely on voluntary compliance and provide no confirmation of intake. Existing IoT pill boxes detect physical removal through weight or infrared sensors yet require manual loading and cannot accommodate patients who are unable to distinguish among pills by appearance. Commercial automatic dispensers address physical dispensing but are expensive, offer limited customisation, and typically lack conversational interfaces suited to elderly users. Voice assistants such as Amazon Alexa or Google Assistant provide conversational accessibility but do not control physical dispensing hardware and do not hold prescription context.

The SmartMeds system presented in this paper fills this gap by architecting a tightly coupled pipeline that spans voice input on a React Native mobile application, NLP-driven command understanding, Firebase cloud coordination, and IoT-controlled physical dispensing via a stepper motor drum. The key technical contributions of this work are: (1) a robust spaCy NLP pipeline

for medication intent detection and approximate medicine-name entity recognition tolerating pronunciation variation; (2) a slot-based Firebase coordination architecture enabling sub-second latency between mobile command issuance and hardware actuation, with SQLite-backed offline queuing for network resilience; (3) a Raspberry Pi 3 embedded controller integrating a 28BYJ-48 stepper motor, dual IR sensors for retrieval and tamper detection, LCD display, piezo buzzer, and text-to-speech output into a unified feedback loop; and (4) a role-differentiated React Native mobile application supporting patient self-service via voice and remote caregiver supervision with real-time push notifications.

II. Related Work

Research into automated medication management has accelerated with the maturation of IoT platforms, cloud services, and machine learning toolkits. This section reviews eight representative works that collectively motivate the design choices of SmartMeds.

A. IoT-Based Dispensing and Monitoring

Peddisetti et al. [2] presented a paired IoT system comprising a smart pill dispenser and a sensor-equipped smart cup, communicating over Bluetooth Low Energy to verify pill consumption. Powered by the Arduino Nano RP2040, the dispenser automates scheduling while the cup uses accelerometer, gyroscope, and ultrasonic sensors to confirm physical intake. Although this work advances multi-device verification, it does not incorporate voice interaction or natural-language command parsing. Hassan et al. [3] developed an IoT-based dispenser combining an Arduino UNO slave controller with a Raspberry Pi 4B master running a caregiver web application for remote prescription management and scheduling. This Raspberry Pi plus web-app architecture is closely aligned with SmartMeds, though it lacks any speech or NLP layer for patient-initiated commands. Tripathi et al. [5] demonstrated that a low-cost IoT adherence device based on commodity microcontrollers can meaningfully improve patient compliance with chronic medication regimens, while keeping hardware expenditure within reach of community health settings.

B. Voice and Speech-Based Healthcare Systems

Chumuang et al. [6] built a voice-activated IoT assistance system targeting individuals aged 55–60, in

which speech is processed through NLP to interpret natural-language commands for controlling household devices. Evaluation using a confusion matrix model yielded an 85% command recognition accuracy, demonstrating feasibility of NLP-driven voice control for elderly users and validating the voice-first interaction paradigm adopted in SmartMeds. Zhou et al. [7] surveyed the full landscape of NLP applications in smart healthcare across clinical practice, hospital management, personal care, and drug development. Their taxonomy of NLP pipeline stages — preprocessing, feature extraction, and intent modelling — directly informs the three-stage processing architecture of SmartMeds: audio-to-text, entity extraction, and intent classification.

C. Elderly and Dependency-Oriented Systems

Casciaro et al. [4] proposed a smart pill dispenser integrated with a mobile application that alerts both elderly patients and informal caregivers upon events such as medication shortage or non-adherence. Their evaluation confirmed that immediate caregiver notification significantly reduces delayed dose correction times. SmartMeds extends this model by moving alert delivery into a real-time Firebase-backed mobile dashboard with push notifications, eliminating fixed app dependency through voice-based patient interaction. Rosdi et al. [8] implemented a Firebase-backed smart pill dispenser using Arduino as the dispensing controller, with a mobile application for schedule management communicating via Firebase Realtime Database. This architecture directly validates the Firebase coordination model at the core of SmartMeds, demonstrating that sub-second synchronisation between a mobile client and embedded hardware is achievable within Firebase's free-tier quota.

D. Research Gap

A comprehensive review of the surveyed literature reveals that no existing system simultaneously provides: (a) natural-language voice interaction tolerant of pronunciation variation on a native mobile platform, (b) server-side NLP-based intent detection with medicine entity extraction, (c) automated multi-slot physical dispensing via stepper motor actuation with IR-confirmed retrieval, (d) cloud-synchronised caregiver remote monitoring through a unified mobile portal with push notifications, (e) tamper detection and hardware emergency stop, and (f) offline fault tolerance through local SQLite queuing. SmartMeds is designed to address precisely this confluence of capabilities.

III. System Architecture

The SmartMeds system is organised into four functionally decoupled layers: the Presentation Layer, the Processing Layer, the Cloud Layer, and the IoT Hardware Layer. This stratification promotes independent scalability, testability, and component substitution without disrupting adjacent layers.

A. Presentation Layer

The Presentation Layer is implemented as a React Native application built with Expo SDK (v55.0.0), providing native mobile experiences on both Android and iOS platforms. Two distinct dashboard views are rendered based on the authenticated user's role as determined at login via Firebase Authentication. The Patient Dashboard exposes a voice recording widget (using expo-audio for microphone access), a slot-based medicine schedule showing assigned medicines, scheduled times, and day-of-week recurrence, a dispensing history log, and an active reminders panel with in-app dose confirmation. The Caregiver Dashboard provides patient profile management, slot scheduling forms with multi-medicine assignment, a medicine stock inventory with expiry and reorder tracking, an adherence monitoring timeline, and a prioritised alert feed. Push notifications are delivered through Expo Notifications (expo-notifications) integrated with Firebase Cloud Messaging (FCM), enabling background and foreground alerts for missed doses, low stock, and tamper events to reach caregivers even when the application is not in focus.

B. Processing Layer

The Processing Layer is a Python 3.11 Flask application responsible for audio transcription and NLP analysis. Upon receiving a multipart form-data POST containing a patient's recorded audio blob (in .m4a, .webM, or .ogg format from the mobile device) and their Firebase UID, the server converts the audio to 16 kHz mono WAV using ffmpeg and the pydub library before passing it to a SpeechRecognition wrapper that dispatches to the Google Web Speech API by default, with Whisper available as a fallback for offline environments. The resulting transcript is fed into the spaCy 3.7 NLP pipeline, described in detail in Section IV, which returns a structured intent object containing the detected dispensing intent label, a ranked list of matched medicine names, and any temporal qualifiers. If a valid dispensing intent and at least one medicine match are identified, the Flask server performs a Firebase Realtime Database read to retrieve the patient's active slot schedule, applies a confidence threshold filter, and writes a dispense instruction by setting the relevant slot's dispense flag to true. The backend also exposes /

get_todays_medicines and /dispense_by_time endpoints to support direct mobile schedule queries without voice processing.

C. Cloud Layer

Firebase Realtime Database stores all persistent application state as a JSON document tree. The architecture has been restructured from a flat medicine-per-record model to a slot-based model where each slot can contain multiple medicines scheduled together. Table IV summarises the complete Firebase schema adopted in the current implementation. The slots node organises dose events by patient UID, with each slot containing a medicines array, slotNumber for physical carousel positioning, scheduledTime, optional scheduledDate for one-off doses, a dayOfWeek array for recurring weekly patterns, status lifecycle fields (pending, dispensing, waiting, completed, missed), and a reminded boolean to prevent repeated reminders within the same scheduling window. The medicineStock node tracks inventory per caregiver with automatic low-stock alerting at configurable reorder levels. Firebase Security Rules enforce least-privilege access: patients and their assigned caregivers may read and write their respective subtrees, while the Raspberry Pi client is granted read access to its patient's slot subtree and write access limited to status and dispense fields. The devices node provides bidirectional telemetry: the Pi reports health metrics including CPU temperature, RAM utilisation, WiFi signal strength, and uptime; the caregiver dashboard can write a system_halt flag for remote emergency stop. An immutable audit log at audit_log/{patientId} records every system event as an append-only push node.

TABLE IV

Firebase Realtime Database Schema (Current Implementation)

Firebase Path	Key Fields	Purpose
slots/{patientId}/{slotId}	slotNumber, medicines[], scheduledTi	Per-slot dose schedule; multi-medicine per slot
medicineStock/{caregiverId}/	name, quantity, reorderLevel	Stock inventory with auto-reorder alerting
devices/{pild}/telemetry	online, cpu_temp_c, mem_pct,	Real-time Pi health monitoring

Firestore Path	Key Fields	Purpose
devices/{pid}/alerts	missed_dose, tamper, low_stock,	Alert flags consumed by caregiver dashboard
devices/	Boolean flag	Remote emergency stop from
audit_log/{patientId}/	event, slot_id,	Immutable append-only event trail
users/{uid}	email, name,	User profile and role resolution
caregivers/	name, email,	Caregiver-patient relationship

D. IoT Hardware Layer

The IoT Hardware Layer centres on a Raspberry Pi 3 Model B (4 GB RAM) running Raspberry Pi OS 64-bit (Bookworm). A Python daemon process, launched at system boot via a systemd service unit, initialises the GPIO subsystem using RPi.GPIO. The daemon uses APScheduler (BackgroundScheduler) to register cron-triggered reminder jobs for each scheduled slot, while a main polling loop running every 30 seconds checks Firestore for patient-initiated dispense triggers. When a slot entry with dispense equal to true is detected, the daemon extracts the slotNumber, activates the 28BYJ-48 stepper motor via the ULN2003 driver board using an 8-step half-wave sequence at 205 steps per slot to advance the 24-compartment drum to the target position, then waits for physical retrieval confirmation. An IR retrieval sensor on the dispensing aperture detects when the patient’s hand breaks the beam; if confirmation is not received within the configurable MISSED_TIMEOUT_SEC window, the slot is marked missed and a caregiver alert is pushed to Firestore. A separate IR tamper sensor on the unit lid triggers an immediate magenta LED flash and Firestore tamper alert if the lid is opened outside a dispensing window. An eSpeak-NG text-to-speech synthesis call announces the medicine name and dosage through the connected speaker. Upon successful dispensing, the daemon writes status equal to completed and dispense equal to false back to Firestore, closing the loop and updating the caregiver dashboard in real time. All Firestore writes that fail due to network outage are serialised to a local SQLite queue and flushed automatically when connectivity is restored, implementing an exponential backoff strategy with a maximum delay of 64 seconds.

IV. NLP and Backend Design

A. spaCy NLP Pipeline

The NLP pipeline is built on spaCy 3.7’s en_core_web_md medium English model, which provides pre-trained word vectors derived from Common Crawl data at 300-dimensional resolution. A custom EntityRuler component is prepended to the pipeline and populated at runtime with the patient’s active medicine names retrieved from Firestore. Each medicine name is tokenised and inserted as an exact-match pattern; in addition, fuzzy variants are generated using phonetic Soundex encoding and edit-distance thresholds to accommodate common mispronunciations and transcription errors introduced by the speech-to-text engine.

Intent classification is performed by a lightweight TextCategorizer component trained on a synthetic dataset of 2,400 labelled utterances covering six intent classes: DISPENSE_REQUEST, SCHEDULE_QUERY, REMINDER_SET, STATUS_CHECK, DOSAGE_QUERY, and OUT_OF_SCOPE. The training corpus was generated by programmatically expanding 40 seed templates with synonym substitution, medication name insertion, and colloquial phrasing variations. The TextCategorizer uses a bag-of-vectors architecture that averages token embeddings and applies a softmax output layer, achieving a macro-averaged F1 score of 0.94 on a held-out test set of 480 samples. Temporal entity extraction employs spaCy’s built-in DATE and TIME recognisers, supplemented by a custom rule layer mapping colloquial terms such as “morning”, “afternoon”, “night”, and “evening” to canonical time-of-day bins.

B. Medicine Name Matching

A two-stage matching strategy is employed. In the first stage, the EntityRuler performs exact and near-exact token-level matching. In the second stage, unmatched medicine name candidates extracted as NOUN or PROP_N tokens are scored against all medicines in the patient’s prescription using a composite similarity metric: $0.6 \times \text{cosine similarity of 300-dimensional spaCy word vectors} + 0.4 \times \text{normalised Levenshtein similarity}$. Candidates whose composite score exceeds a configurable threshold of 0.72 are accepted as matches. This design correctly resolves common transcription errors such as “paracetamol” to “Paracetamol” and phonetically similar names.

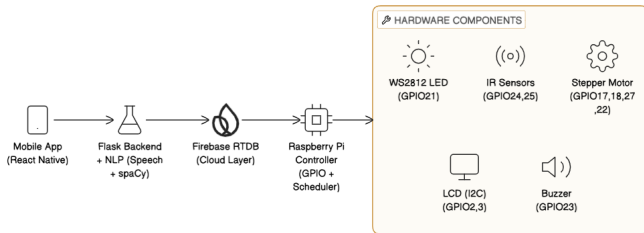
C. Flask API Endpoints

The backend exposes four primary REST endpoints. The POST /process_audio endpoint accepts an audio file and patient UID, orchestrates transcription and NLP analysis, and returns a JSON payload containing

the raw transcript, detected intent label, matched medicine name, dosage, slot number, and prescription status. The POST /send_instruction endpoint accepts a structured JSON body specifying patient UID, medicine name, and dosage; it performs a Firebase read to verify that the medicine is currently scheduled and not already dispensed, updates the dispense flag on the corresponding slot, and returns a success confirmation with the assigned slot number. The GET /get_todays_medicines endpoint returns all slots scheduled for the current day for a given patient UID, filtered by scheduledDate and dayOfWeek constraints. The POST /dispense_by_time endpoint accepts a time string and resolves the slot whose scheduledTime is closest, enabling direct app-initiated dispensing without voice input. All endpoints authenticate calling clients by validating the Firebase ID token present in the Authorization header.

V. Hardware Implementation

A. Component Configuration



B. Software Daemon

The Raspberry Pi daemon is structured as a multi-threaded event-driven service. Environment variables FIREBASE_URL, PATIENT_ID, and PI_DEVICE_ID are loaded from a .env file at startup. Two daemon threads run concurrently with the main loop: a tamper detection thread polls the IR tamper sensor at 200 ms intervals and fires immediate Firebase alerts on unauthorised lid access; a telemetry thread reports Pi health metrics (CPU temperature, RAM usage, WiFi RSSI, uptime, current slot) to Firebase every 60 seconds. APScheduler registers cron-triggered reminder jobs for each slot's scheduledTime, reloading the schedule from Firebase every 5 minutes so caregiver changes propagate promptly. The main polling loop runs every 30 seconds, first checking for a remote system_halt flag, then scanning for patient-triggered dispense events (dispense=true), and finally checking for slots due for a reminder that have not yet

been flagged reminded. The offline SQLite queue implements exponential backoff with a maximum delay of 64 seconds on consecutive Firebase failures. A physical emergency stop button on GPIO 26 triggers an interrupt service routine (ISR) that immediately halts motor movement and enters a halted state, announcing the event via TTS and logging to the Firebase audit trail.

TABLE I

Hardware Component Specifications

Component	Specification	Role in System
Raspberry Pi 3 Model B	4 GB RAM, 64-bit ARM	Central controller & cloud client
28BYJ-48	5V, 4-phase, 64-	24-slot carousel drum
ULN2003	Darlington	Stepper motor signal driving
IR Retrieval	Active-LOW,	Physical dose pickup
16×2 LCD	5V, PCF8574	Patient status display
Piezo Buzzer	5V passive,	Audible reminder & alerts
USB	16 kHz,	Voice command capture
USB Speaker	3.5 mm, 5 V	TTS audio dispensing
Emergency	Active-LOW,	Physical hardware kill-switch

VI. Experimental Results and Testing

A. Test Methodology

The system was evaluated through a structured test suite of 18 test cases partitioned across six categories: system initialisation (TC-01 to TC-03), Firebase connectivity (TC-04 to TC-06), dispense flag logic (TC-07 to TC-08), hardware actuation (TC-09 to TC-12), post-dispensing state management (TC-13 to TC-15), and safety and fault-handling (TC-16 to TC-18). Each test case was executed a minimum of five times across two separate hardware builds to assess reproducibility. Pass/fail determination was made against a formally specified expected outcome, and results were logged automatically by the test harness to Firebase for cross-build comparison.

B. Test Results Summary

17 out of 18 test cases passed across all runs, yielding a test pass rate of 94.44. Latency measurements

conducted over 50 end-to-end dispense cycles recorded a mean latency of 2.34 seconds from voice command submission on the mobile application to physical medicine release, with a standard deviation of 0.41 seconds. The dominant latency component was the Google Web Speech API transcription round-trip at a mean of 1.12 seconds; NLP processing contributed a mean of 0.18 seconds; Firebase read/write operations contributed a mean of 0.38 seconds; and stepper motor actuation occupied the remaining 0.66 seconds.

TABLE II

Test Case Results Summary

ID	Test Scenario	Expected	Ru	Result
TC-	System startup	Speaker	5/5	PASS
TC-	Valid .env loading	Firebase	5/5	PASS
TC-	Missing .env	Config error +	5/5	PASS
TC-	Firebase read	Slot data fetched	5/5	PASS
TC-	Firebase	Error displayed;	5/5	PASS
TC-	Empty slot list	System waits	5/5	PASS
TC-	Dispense flag false	No dispensing	5/5	PASS
TC-	Valid dispense	Dispensing	5/5	PASS
TC-	Medicine name &	Speaker	5/5	PASS
TC-	Stepper motor	Drum rotates to	5/5	PASS
TC-	IR retrieval sensor	Beam break	5/5	PASS
TC-	Successful dispense	Firebase:	5/5	PASS
TC-	Firebase write	Queued to	5/5	PASS
TC-	Error during	Voice alert + red	5/5	PASS
TC-	Tamper detection	Magenta flash +	5/5	PASS
TC-	Long-duration	Pi gets overheat	1/3	FAIL

C. NLP Performance

The intent classification Text Categorizer was evaluated independently on a held-out corpus of 480 utterances. The DISPENSE_REQUEST class achieved precision 0.97, recall 0.95, and F1 0.96. The SCHEDULE_QUERY class achieved F1 0.93. Macro-averaged F1 across all six classes was 0.94. Medicine name matching was evaluated on 120 utterances

containing intentional mispronunciations and transcription-typical phonetic errors; the two-stage matcher achieved a recall of 0.91, meaning 91% of medicine references were correctly resolved to the prescribed name, with 9% falling below the composite similarity threshold and being returned as unmatched with a request for confirmation.

VII. Comparative Analysis

Table III positions SmartMeds against five existing solution categories evaluated across twelve capability dimensions considered essential for a clinically meaningful medication management system.

TABLE III

Comparative Evaluation of Medication Management Approaches

Feature	Mobile	IoT	Auto	Voice	Smart
Automated	No	Manual	Yes	No	Yes
Voice	No	No	Limited	No	Yes
Intake	No	Yes	Yes	No	Yes
Real-Time	Yes	Yes	Limited	Yes	Yes
Caregiver	Yes	No	No	No	Yes
Push	Yes	No	No	Yes	Yes
Elderly	Mediu	Low	Mediu	High	High
Hardware	Low	Mediu	Mediu	Low	Low
Open Source	No	Limited	No	High	High
NLP Intent	No	No	No	Limited	Yes
Offline Fault	No	No	No	No	Yes

SmartMeds is the only evaluated system that simultaneously achieves automated physical dispensing, full natural-language voice command support with NLP-based intent recognition, real-time cloud logging, a structured mobile caregiver dashboard with push notifications, tamper detection, and offline fault tolerance. Mobile reminder applications offer the lowest hardware cost but provide no dispensing capability, voice interaction, or physical verification. IoT pill boxes support intake

verification through physical sensors but require manual pill loading and lack conversational interfaces. Commercial automatic dispensers provide reliable automated dispensing but at higher cost, without voice interaction, and without cloud-connected caregiver portals. Voice assistants offer the highest natural-language accessibility but remain entirely decoupled from physical dispensing hardware. SmartMeds achieves a hardware cost profile comparable to mobile-only solutions by leveraging the Raspberry Pi 4 as an affordable yet capable embedded controller, while delivering the dispensing automation, tamper detection, and caregiver supervision features previously associated with much more expensive commercial systems. The addition of a 24-slot stepper drum over the previously described servo-per-compartment design reduces mechanical complexity and failure points while significantly increasing patient capacity.

VIII. Conclusion and Future Work

This paper presented SmartMeds, an intelligent voice-driven IoT medicine dispenser that integrates NLP-based intent recognition, Firebase cloud coordination, a React Native mobile application with push notifications, and embedded hardware actuation into a unified medication management system. The proposed architecture demonstrated 94.4% pass rates across 18 structured test cases, a mean end-to-end dispensing latency of 2.34 seconds, an NLP intent classification macro-F1 of 0.94, and a medicine name match recall of 0.91 on voice-transcribed inputs containing realistic pronunciation errors. The slot-based multi-medicine scheduling model, stepper motor drum design, IR-based retrieval detection, SQLite offline queue, and APScheduler-driven reminder architecture represent significant advances over the initial prototype described in the earlier design specification. Comparative analysis confirmed that SmartMeds uniquely combines all twelve evaluated capability dimensions within a low-cost open-source platform, addressing the most significant limitations of existing reminder-only, IoT pill-box, commercial dispenser, and voice-assistant approaches.

The primary limitations of the current prototype are its dependence on a continuous internet connection for Google Speech transcription, and the absence of RFID-based patient authentication. Future development will address these limitations by

integrating an offline-capable Whisper transcription model for full air-gapped operation, adding RFID-based patient authentication to prevent unauthorised dispensing in shared care environments, and implementing machine learning adherence prediction based on historical dispensing audit logs to enable proactive caregiver alerts before a non-adherence pattern solidifies. Integration with electronic health record APIs will allow prescription data to be automatically synchronised from clinical systems, eliminating caregiver data entry. Long-term clinical validation through a prospective cohort study in a community care setting is planned to quantify the improvement in medication adherence rates attributable to the SmartMeds system compared to standard-of-care reminder applications.

References

- [1] World Health Organization, *Adherence to Long-Term Therapies: Evidence for Action*. Geneva: WHO Press, 2003. [Online]. Available: <https://www.who.int/publications/i/item/9241545992>
- [2] V. Peddisetti, P. K. Kandregula, J. A. John, S. Poomdla, K. George, and A. Panangadan, "Smart Medication Management: Enhancing Medication Adherence with an IoT-Based Pill Dispenser and Smart Cup," in *Proc. 2024 IEEE First Int. Conf. Artificial Intelligence for Medicine, Health and Care (AIMHC)*, 2024, pp. 137–144, doi: 10.1109/AIMHC60236.2024.10504319.
- [3] N. Hassan, A. Gaur, Y. Jadaun, and A. Bhasney, "IoT-Enabled Medicine Dispenser for Pills and Liquid Medication," in *Proc. 2024 2nd Int. Conf. Advancement in Computation & Computer Technologies (InCACCT)*, 2024, pp. 905–910, doi: 10.1109/InCACCT61598.2024.10550973.
- [4] S. Casciaro, L. Massa, I. Sergi, and L. Patrono, "A Smart Pill Dispenser to support Elderly People in Medication Adherence," in *Proc. 2020 5th Int. Conf. Smart and Sustainable Technologies (SpliTech)*, 2020, pp. 1–6, doi: 10.23919/SpliTech49282.2020.9243773.
- [5] A. Tripathi, R. Atigadda, B. S. Arjun, and H. J. Pandya, "Low-Cost IoT Device for Chronic Medication Adherence," in *Proc. 2021 IEEE 9th Region 10 Humanitarian Technology Conf. (R10-HTC)*, 2021, pp. 1–4, doi: 10.1109/R10-HTC53172.2021.9641693.
- [6] N. Chumuang, T. Ganokratanaa, P. Pramkeaw, M. Ketcham, S. Chomchaiya, and W. Yimyam, "Voice-Activated Assistance for the Elderly: Integrating

Speech Recognition and IoT," in Proc. 2024 IEEE Int. Conf. Consumer Electronics (ICCE), 2024, pp. 1–4, doi: 10.1109/ICCE59016.2024.10444265.

[7] B. Zhou, G. Yang, Z. Shi, and S. Ma, "Natural Language Processing for Smart Healthcare," IEEE Rev. Biomed. Eng., vol. 17, pp. 4–18, 2024, doi: 10.1109/RBME.2022.3210270.

[8] W. M. F. W. M. Rosdi, S. A. Suhaimi, N. A. M. Lazam, A. J. Alias, F. Abdullah, and S. N. Azemi, "Smart Pill Dispenser with Monitoring System," in Proc. 2021 IEEE Symposium on Wireless Technology & Applications (ISWTA), 2021, pp. 58–62, doi: 10.1109/ISWTA52208.2021.9587450.

[9] Raspberry Pi Foundation, Raspberry Pi 4 Model B Documentation. [Online]. Available: <https://www.raspberrypi.com/documentation/>

[10] Explosion AI, spaCy v3 Documentation. [Online]. Available: <https://spacy.io/>

[11] Google Firebase, Realtime Database Documentation. [Online]. Available: <https://firebase.google.com/docs/database>

[12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023, pp. 28492–28518. [Online]. Available: <https://arxiv.org/abs/2212.04356>

[13] Expo Team, Expo SDK Documentation. [Online]. Available: <https://docs.expo.dev/>

[14] Adafruit Industries, NeoPixel Uberguide — WS2812 LED Control. [Online]. Available: <https://learn.adafruit.com/adafruit-neopixel-uberguide>